Sanitize Data for AI Analysis

Scenario

Data given to AI models is rarely perfect. Raw datasets often contain messy formatting, missing values, invalid entries, duplicates, and even sensitive information. If left unaddressed, these problems lead to poor AI outputs, wasted compute cycles, and inaccurate insights.

In this lab, you'll learn how to sanitize and validate data so it's ready for AI-driven analysis. You'll explore the limitations of raw “dirty” data, practice hands-on cleaning, and then apply automation and Open WebUI's knowledge features to make your cleaned dataset reusable.

Your Mission

- Compare how an AI performs with raw vs. cleaned data.
- Practice manual cleaning of common issues.
- Use Python and pandas to automate large-scale sanitization.
- Add the cleaned dataset to an Open WebUI knowledge store and query it effectively.

Pre Requisites

Download the files from labfile3.1 from the following GitHub repository:

https://github.com/InfosectrainSecAI/CompTIASecAI

And save it in home/kali/Desktop/labfile3.1

Part 1: Loading Raw Data into Open WebUI

Step 1: From the desktop, open **Firefox**. Open and sign into **Open WebUI**.

Step 2: Select **Workspace** from the left menu, then select the **Knowledge** tab.

- If the menu is not visible, select the three horizontal lines to the left of the model name to activate the menu.

Step 3: Select the **+** to the right of the page to add a new knowledge base.

Step 4: On the **Create a knowledge base** page, set the name to **Customer Data**, set the description to **Data Cleaning**, and set the visibility to **Public**. Select **Create Knowledge**.

> Insights: In Open WebUI, the Knowledge Store lets you upload documents or datasets for the model to use directly in chats. Instead of pasting large files into a prompt, you attach them to a knowledge workspace once, and then reference them in your conversations.

Step 4: From the **Customer Data** knowledge store, select **+** to the right of the page, then select **Upload files**.

Step 5: Select **dirty_data.csv** from **/home/kali/Desktop/labfile3.1** and select **Open**.

> The file will be added to the collection on the right.

Step 6: With the data uploaded, select **New Chat** from the top left of OpenWebUI.

Step 7: In a new chat with **gemini**, enter **#** to bring up the knowledge store. Select **dirty_data.csv** from the list.

Step 8: In the chat, enter the following prompt and note the response:

```
Prompt 1: Please list all customer SSNs.
```

> Insights: For security reasons, the model may refuse to provide the requested output. The model output may differ slightly in verbosity, but the refusal to act is the same.

Step 9: In the same chat, enter the following prompt and note the response:

```
Prompt 2: Show the customers that have SSNs listed.
```

> Insights: At first the model refused to give raw SSNs, but when the prompt was reworded it did return rows containing SSN values. That shows how fragile and inconsistent model behavior can be: small wording changes may produce very different outputs. Use this as a cautionary example — don't rely on the model to enforce privacy; sanitize your data upstream and apply consistent controls.

Step 10: In the same chat, enter the following prompt and note the response:

```
Prompt 3: List the email and phone number for all customers from Chicago.
```

> Insights:When you prompt a model with personal or structured data, it will attempt to represent that data as given — including any errors, inconsistencies, or missing data. If the input isn't consistent or secure, those flaws are likely to show up in the model's output as well.

Manual Cleaning and Validation

Cleaning data by hand isn't scalable, but it's the fastest way to understand common issues. In this step, you'll fix a few sample problems while learning how to spot and standardize data inconsistencies.

Step 1: Minimize OpenWebUI and return to the **Kali** desktop.

Step 2: Open the **labfile3.1** folder from **/home/kali/Desktop**, then open **dirty_data.csv**.

> Action: The file will open in LibreOffice Calc. Select OK on the Text Import window.

Step 3: In the **dirty_data.csv** file, select **row 7**, right-click and select **Delete Rows**.

> Insights: Row 7 contains no data and only adds unnecessary whitespace to the dataset. Extra empty rows can also confuse the model's output, depending on how the prompt is phrased.

Step 4: Select cell **B4** and replace **MARIA** with **Maria**.

> Insights: Consistent grammar, capitalization, and punctuation are essential for clean datasets. For AI specifically, messy formatting increases confusion, reduces accuracy, and may even surface duplicate or incomplete results. Consistency ensures the model interprets each record as intended.

Step 5: Select cell **D13** and replace **matthew@** with **Missing or Incomplete**.

> Insights: This is one of the results from our earlier prompt: Matthew Flores' email address is missing its domain. When information is missing or incomplete, it's important to clearly mark it as such rather than leaving it blank or partially filled. This makes it obvious to anyone reviewing the dataset that the value isn't valid, and it also prevents confusion during searches or when prompting AI models.

Step 6: Select cell **J2** and replace **387726868** with *****-***-6868**.

> Insights: Earlier in this lab, you saw how the model willingly surfaced Social Security Numbers when they weren't masked. That happened because the data was raw and the model simply echoed back what it found. This highlights why masking is so important: if sensitive values are left exposed, they can slip into outputs even when you don't explicitly ask for them.

As you've just experienced, cleaning data by hand can be tedious and error-prone. Correcting capitalization, fixing phone formats, masking SSNs, and filling in missing fields one by one is not only time-consuming, it also increases the chance of human mistakes slipping in. This is why automation matters. By using scripts and tools, you can apply consistent rules across an entire dataset in seconds.

In the next exercise, you'll see how a Python script can take care of these repetitive tasks all at once — giving you cleaner, safer data with just a single run.

You've successfully performed manual data cleaning and validation.

Automated Data Cleaning with Python

Manual cleaning doesn't scale. To handle larger datasets, you'll use a Python helper script that applies sanitization rules systematically with pandas.

Step 1: Close the **dirty_data.csv** file and open a terminal from the left menu.

> **⚠ Important Note: Do NOT save the changes you made to dirty_data.csv in the previous step.**

Step 2: Open the terminal and run the following commands to call the helper script:

```
cd /home/kali/Desktop/labfile3.1/
```

```
python3 clean.py
```

> Insights: Instead of fixing records by hand, pandas lets you define these cleaning rules once and apply them across every row automatically, producing a dataset that's more accurate, consistent, and secure in just one run.

Step 3: Close the terminal and open the **cleaned_data.csv** file from the **labfile3.1** folder.

> Action: The file will open in LibreOffice Calc. Select OK on the Text Import window.

Step 4: Review the cleaned and secured data.

Issues the Script Fixes:

Issue Fixed	How It's Handled
Capitalization & formatting	Standardizes names, cities, and addresses.
Date of birth	Converts valid dates to YYYY-MM-DD format, marks invalid entries as Missing or Incomplete.
Phone numbers	Normalizes into the format (XXX) XXX-XXXX.
Email addresses	Invalid or incomplete emails marked as Missing or Incomplete.
SSNs & Credit Cards	Masked to show only the last four digits.
Bank accounts	Masked except for the last four digits.
Passwords	Replaced with [REDACTED].
Missing data	Clearly marked as Missing or Incomplete instead of blanks.
Duplicates & empty rows	Removed to prevent confusion.

The cleaning script uses Python together with the pandas library to quickly scan and fix issues in the dataset. It applies consistent rules to each column: standardizing names, dates, and phone numbers, masking sensitive fields like SSNs and credit card numbers, filling in missing values with
a clear placeholder, and dropping empty or duplicate rows.

Testing Clean Data

Now that you've sanitized the dataset, it's time to see the difference it makes. In this step, you'll upload the cleaned file into the Open WebUI knowledge store and run the same types of prompts you tried earlier against the raw data. By comparing the two results side-by-side, you'll see how much easier it is for the model to provide accurate, consistent answers once the data has been standardized and sensitive fields have been masked.

Step 1: Return to **OpenWebUI** and select **Workspace** from the left menu.

Step 2: Select the **Knowledge** tab, then select the **Customer Data** collection.

Step 3: From the **Customer Data** knowledge store, select **+** to the right of the page, then select **Upload files**.

Step 4: Select **cleaned_data.csv** from **/home/kali/Desktop/labfile3.1** and select **Open**.

Step 5: Select **New chat** from the upper-left.

Step 6: In a new chat with **gemini**, enter **#** to bring up the knowledge store. Select **cleaned_data.csv** from the list.

Step 7: With the cleaned data loaded, enter the following prompt and note the response:

```
Prompt 1: Show the customers that have SSNs listed.
```

> Insights: Now that the SSNs have been masked, the model may not show them at all in its response. However, ensuring sensitive data is secured can prevent unexpected leaks.

Step 8: In the same chat, enter the following prompt and note the response:

```
Prompt 2: Show me all customers with a missing or incomplete emails.
```

Insights: When the dataset has blanks, "N/A"s or partial entries, the model doesn't have a consistent
standard for what counts as missing or incomplete. Because of this, it may treat some records differently each time, leading to varied answers. By standardizing all such cases as
Missing or Incomplete, the cleaned dataset removes that ambiguity and allows the model to give consistent, reliable results.